

Intermediate (swap in a structured parser) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `structured_output_practice.py` (Intermediate section):

`.with_structured_output(Model)` is LangChain's version of the exact same constrained-decoding call Doc04 made directly against the SDK — this exercise is where you feel that they're the same idea, reached two different ways. **If not:** the Build Task's parser choice would be the first time you ever saw LangChain's structured-output call, with no smaller version to compare it against Doc04's raw one.

Basic Version

Approach 1 — the direct way

```
# structured_output_practice.py – Intermediate section
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

class Person(BaseModel):
    name: str
    age: int

structured_model = ChatOpenAI().with_structured_output(Person)

good_result = structured_model.invoke("Extract: Maria is 34 years old.")
print("Good input:", good_result)

try:
    bad_result = structured_model.invoke("Extract: the sky is blue today.")
    print("Bad input:", bad_result)
except Exception as e:
    print("Caught error:", type(e).__name__, "-", e)
```

Expected output:

```
Good input: name='Maria' age=34  
Caught error: ValidationError - 1 validation error for Person...
```

This version works correctly and shows the difference clearly. It catches the broad `Exception` instead of the specific validation error class, which is fine for exploring, but not what you'd want in real code.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — catching the specific `ValidationError`

```
# structured_output_practice.py – Intermediate section
from pydantic import BaseModel, ValidationError
from langchain_openai import ChatOpenAI

class Person(BaseModel):
    name: str
    age: int

def get_structured_model() -> ChatOpenAI:
    # why: .with_structured_output(Person) is LangChain's version of the
    # same constrained-decoding call Doc04 made directly against the SDK.
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return model.with_structured_output(Person)

def main() -> None:
    structured_model = get_structured_model()

    good_result = structured_model.invoke("Extract: Maria is 34 years old.")
    print(f"Good input -> {good_result!r}")

    try:
        structured_model.invoke("Extract: the sky is blue today.")
    except ValidationError as e:
        # how: catching the specific error first means a real "didn't
        # match my shape" failure is distinguishable from anything else.
        print(f"Caught a validation error, as expected: {e}")
    except Exception as e:
        print(f"Caught an unexpected error type: {type(e).__name__}: {e}")

if __name__ == "__main__":
    main()
```

Expected output:

```
Good input -> Person(name='Maria', age=34)
Caught a validation error, as expected: 1 validation error for Person...
```

Difference from Basic: catching the specific `ValidationError` (from `pydantic`) first, with a broader `Exception` fallback after it, matches Doc01's "catch the specific error you know how to handle" rule — in real code, you want to react differently to "the model's answer didn't match my shape" than to, say, a network timeout, and a single broad `except Exception` can't tell them apart.

Which one should you write? Intermediate's catch-and-report is enough for exploring, or for a feature where a failure should stop the whole operation — which is what this document's Build Task needs. Two real options worth knowing exist for a mismatch you don't want to be fatal:

`.with_fallbacks([...])` swaps in a plain-text chain so the call never raises (the caller then checks `isinstance(result, Person)`), or retry with the validation error fed back into the prompt, asking the model to try again. Neither is required here — reach for them once a structured-output failure needs to degrade gracefully instead of stopping the operation.